

Sistema de gestión del V Congreso Argentino de Agroecología

Trabajo Final Integrador — **Grupo 1**

De la maqueta al sistema desplegado: seis entregas, una aplicación en producción.

Integrantes: Benjamín Rodríguez Mantilla · Alcides Castro · Lucas Budnik

Jakarta EE 11

Jersey 3.1

Hibernate 7 · MySQL

Angular 19

Docker + GitLab CI

01

El problema

Qué congreso hay que gestionar, cómo nos acercamos y para quién construimos.

Un congreso no es un formulario: son cinco procesos en paralelo

2 min

El **V Congreso Argentino de Agroecología** (FCAyF–UNLP) reúne durante varios días a cientos de personas: docentes, productores, estudiantes y organizaciones. Hoy eso se coordina con planillas, mails y formularios sueltos.

Inscribirse y pagar

Categorías con aranceles distintos, transferencia o efectivo, comprobantes, recibos y facturas.

Enviar y evaluar trabajos

Resúmenes por eje temático, revisión por pares a ciegas, dictámenes, desempates y devoluciones.

Armar el programa

Mesas, pósters, talleres y conferencias en aulas con cupo y franjas horarias que no pueden solaparse.

Comunicar

Circulares, avisos por etapa y recordatorios: hoy, listas de correo copiadas a mano.

Certificar

Asistencia, presentación y evaluación. Al cierre, cada persona debe poder descargar lo suyo.

El costo de no tenerlo

Datos duplicados, plazos que se pasan, trabajos sin evaluador y certificados armados uno por uno.

Cómo nos acercamos al problema

- **Partimos del congreso real.** Del primer encuentro sobre el V Congreso Argentino de Agroecología 2027 y de lo documentado en ediciones anteriores (crecimiento de asistentes y trabajos, problemas con mails y planillas), para entender qué necesitaba realmente el sistema.
- **Primero un evento fijo; después plazos del comité.** Modelamos un congreso de tres días. Ahí vimos que casi todo lo que el sistema permite o bloquea depende del momento: inscripción, envío, evaluación, certificados. Pensando en otras ediciones, esos plazos los hicimos configurables: que dependan de la gestión del comité organizador, no de fechas fijas en el código.
- **De la organización real a roles, épicas e historias.** Vimos que el congreso necesitaba mejor organización porque hoy distintas personas hacen **tareas a mano** (inscribir y cobrar, enviar y evaluar trabajos, armar el programa, comunicar). Eso definió **roles concretos**, no un usuario genérico. Con esa base armamos la solución por **épicas** —es decir, grandes módulos del sistema: usuarios, inscripción, trabajos, cronograma, comunicación— y, dentro de cada una, **historias de usuario** (qué quiere hacer cada rol y para qué). Antes de programar las probamos en un prototipo navegable, solo de interfaz.

Usuarios y adoptantes de la solución

6 roles

Salieron de las tareas que hoy se hacen a mano: de ahí definimos roles y armamos la solución por épicas (módulos) e historias de usuario.

ROL	QUÉ HACE EN EL SISTEMA	POR QUÉ LE IMPORTA
Asistente	Se crea una cuenta e inicia sesión · se inscribe al congreso y completa sus datos · paga la inscripción (transferencia o efectivo) y sube el comprobante · consulta si su pago fue aprobado · arma su agenda personal del congreso sin chocar horarios · puede proponer un taller · descarga su certificado cuando el congreso termina.	Sabe en todo momento si su inscripción está bien, sin mandar mails para preguntar.
Autor	Además de lo del asistente: carga un trabajo (artículo o relato) con coautores y eje temático · sube el PDF · lo guarda como borrador o lo envía a evaluación · ve en qué estado está y lee la devolución · cuando lo aceptan, ve cuándo y dónde presenta.	Sigue su trabajo de punta a punta y recibe la devolución sin intermediarios.
Evaluador	Recibe trabajos de su eje temático · acepta o rechaza la asignación · lee el PDF · da su dictamen (aceptar, rechazar u observaciones) · puede adjuntar una corrección · no evalúa trabajos propios.	Solo recibe trabajos de su especialidad, con un cupo claro y sin conflicto de interés.
Comité académico (organizador científico)	Hace la prevalidación de los trabajos (revisión inicial antes de asignar evaluadores) · asigna evaluadores por eje temático respetando cupos · resuelve empates y reenvíos · aprueba solicitudes de nuevos evaluadores · configura ejes, modalidades y plazos de envío · consulta estadísticas del proceso académico.	Es el adoptante principal : hoy hace todo esto a mano.
Administración	Da de alta y gestiona usuarios · define aranceles (cuánto paga cada categoría de inscripción) · valida pagos, emite recibos/facturas y hace el arqueo de caja (control de lo cobrado en el día) · carga aulas, horarios y el programa del congreso · publica circulares y avisos · cierra el congreso y habilita certificados · ve reportes y pendientes de organización.	Centraliza la operación del evento y el control del dinero.
Público	Sin cuenta puede ver la información general, el programa publicado, las circulares, la historia del congreso y el mapa/sede · desde ahí puede registrarse o iniciar sesión.	Es la cara visible del congreso: informarse sin barreras.

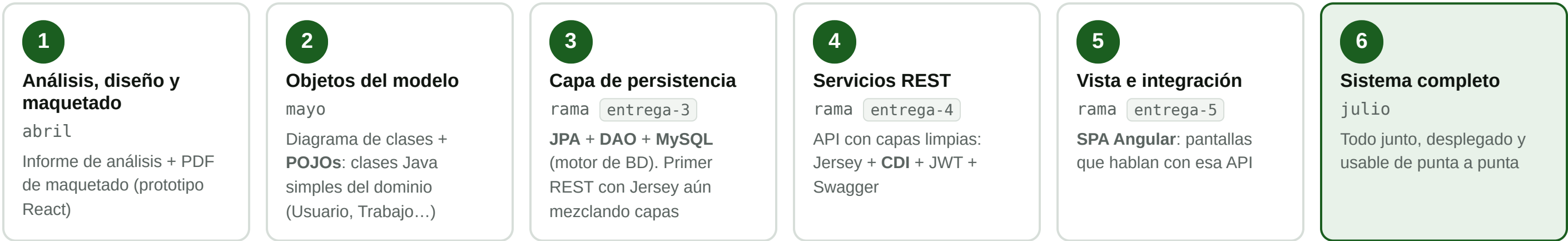
02

Las seis entregas

Qué objetivo tuvo cada etapa, qué conceptos de la cátedra aplicamos, con qué nos peleamos y qué quedó funcionando.

El camino: de la maqueta al sistema desplegado

abril → julio 2026



Una regla que nos salvó

Cada entrega vivió en su propia rama (entrega-3 , entrega-4 , entrega-5) y se integró a main sólo con el pipeline en verde. Nunca perdimos una entrega ya aprobada por trabajar sobre la siguiente.

Análisis, diseño y maquetado

Producto: informe + PDF de maquetado

Objetivo

Entender el dominio del V Congreso y acordar *qué* construir, con pantallas discutibles. Relevamiento → roles → **iniciativa** → **épicas** → **HU** → prototipo cliente–servidor (aún sin CDI).

① Análisis — HU más importantes para el congreso

- **HU-01 / HU-02** — Registrarse e iniciar sesión (entrada al sistema).
- **HU-06** — Solicitar inscripción y adjuntar comprobante de pago.
- **HU-07** — Admin valida pagos (aprobar/rechazar) y habilita al asistente.
- **HU-11** — Enviar un trabajo (PDF, eje, oral/póster) a evaluación.
- **HU-12** — Evaluador dictamina (aprobar/rechazar + devolución opcional).
- **HU-13 / HU-14** — Admin crea actividades y publica el programa.
- **HU-17** — Asistente arma su agenda personal sin solapar horarios.

Salen del informe *Análisis del Problema Sobre el V Congreso* (épicas 1–5).

Dificultad

Al maquetar aparecieron reglas que no estaban escritas: ¿qué pasa si dos evaluadores empatan?, ¿puede un evaluador revisar su propio trabajo?

Cómo la resolvimos: las anotamos en el análisis / HU y las llevamos al modelo de la Entrega 2.

② **Maquetado:** `archivo_maquetado.pdf` — capturas del prototipo **React + Vite** (`localStorage`, sin backend). Siguiente diapositiva: pantallas ligadas a esas HU.

Entrada al congreso: registro, inscripción y validación

archivo_maquetado.pdf

22/4/26, 13:13

Untitled

V Congreso Argentino de Agroecología

La Plata 2027

Iniciar sesión

Registrarse

👤

Registrarse

Crea tu cuenta para participar

Nombre

Apellido

Email

Contraseña

Confirmar contraseña

Registrarse

¿Ya tienes cuenta? Iniciar sesión

HU-01 Registro — crear cuenta para entrar al sistema

V Congreso Argentino de Agroecología

La Plata 2027

julieta

📄

Inscripción al Congreso

Completá tus datos y adjuntá el comprobante de pago

Institución

Provincia

Selecciona

Categoría de Inscripción

Selecciona una categoría

Comprobante de Pago

⬆

Seleccionar archivo Sin archivos seleccionados

HU-06 Inscripción — datos + subir comprobante de pago

V Congreso Argentino de Agroecología

La Plata 2027

Admin

Panel de Administración

Gestión completa del congreso

Crear Mesa Temática

Crear Mesa Redonda

Crear Sesión de Pósters

Crear Taller

Crear Conferencia

Programa publicado

Controla si el cronograma es visible para el público en "Programa".

Publicado

Validación de Inscripciones

julieta lopez

Ver comprobante

Aprobar Rechazar

HU-07 Admin — ver comprobante, Aprobar / Rechazar

Entrega 1 · capturas 1/2

9 / 29

Trabajos, evaluación y programa

archivo_maquetado.pdf

V Congreso Argentino de Agroecología

1 julieta

La Plata 2027

Envío de Trabajos

Trabajos enviados: 1

Título

Eje temático

Oral

Poster

Adjunto (PDF obligatorio)

Seleccionar archivo Sin archivos seleccionados

El archivo se guarda localmente en este navegador para su revisión.

Enviar trabajo

HU-11 Envío — título, eje, oral/póster, PDF, Enviar

Evaluados

0

Aprobados

0

Trabajos Propuestos

Pendientes: 1

Red de Facilitadores en Agroecología

poster

Relato de experiencia

Pendiente

Ver / descargar PDF

Agregar comentario (opcional)

Aprobar

Rechazar

HU-12 Evaluador — PDF, comentario, Aprobar / Rechazar

PROGRAMA

Este es el calendario de actividades: mesas temáticas, mesas redondas, pósters y talleres.

Desde 10 de may de 2027 — hasta 12 de may de 2027

Ver programa completo

HU-13/14 Programa — calendario de actividades

Prototipo sin backend: validar el flujo de las HU antes de modelar en Java.

Entrega 1 · capturas 2/2

10 / 29

¿Cuál es nuestro diseño?

modelo de objetos del dominio

Diseñamos el congreso como **objetos Java del negocio** (POJOs): cada cosa importante es una clase con datos, estado y relaciones. Ese diseño **no guarda nada en MySQL todavía** — solo dice *qué* existe.

Ejemplo concreto

Trabajo

título, resumen, estado, PDF... métodos como `enviar()`.

AsignacionEvaluacion

¿este evaluador aceptó revisar este trabajo?

Evaluacion

el dictamen: aprobar / rechazar + comentario.

Lo mismo para `Usuario`, `Pago`, `InscripcionCongreso`, `Actividad` ... Más adelante mostramos ese diseño en el diagrama completo.

Qué problema deja planteado

Cuando el servidor se apaga, esos objetos en memoria **se pierden**.

Hace falta **persistencia**: copiar cada objeto a una fila de MySQL y volver a armarlo después.

Pregunta de diseño: ¿quién se ocupa de guardar y buscar? Si lo metemos en `Trabajo` o en el servlet, mezclamos el negocio con MySQL.

Por eso elegimos el patrón DAO: una responsabilidad aparte (acceso a datos de una entidad). Siguiendo: por qué ese camino.

Por qué fuimos por DAOs

separar “el negocio” de “la base”

El modelo de objetos ya estaba partido en entidades claras (`Usuario` , `Trabajo` ...). Eso nos empujó a un camino simple: **una entidad del diagrama → una tabla → un DAO**.

Camino que evitamos

Meter SQL / `EntityManager` **adentro** de `Usuario` o del servlet:

- `Usuario` sabría de MySQL → si cambia la BD, tocás el negocio.
- El mismo `persist` / `find` se copia en 10 pantallas.
- No se puede testear el dominio sin levantar la base.

Mezcla dos trabajos: “reglas del congreso” + “hablar con MySQL”.

Camino que elegimos: DAO

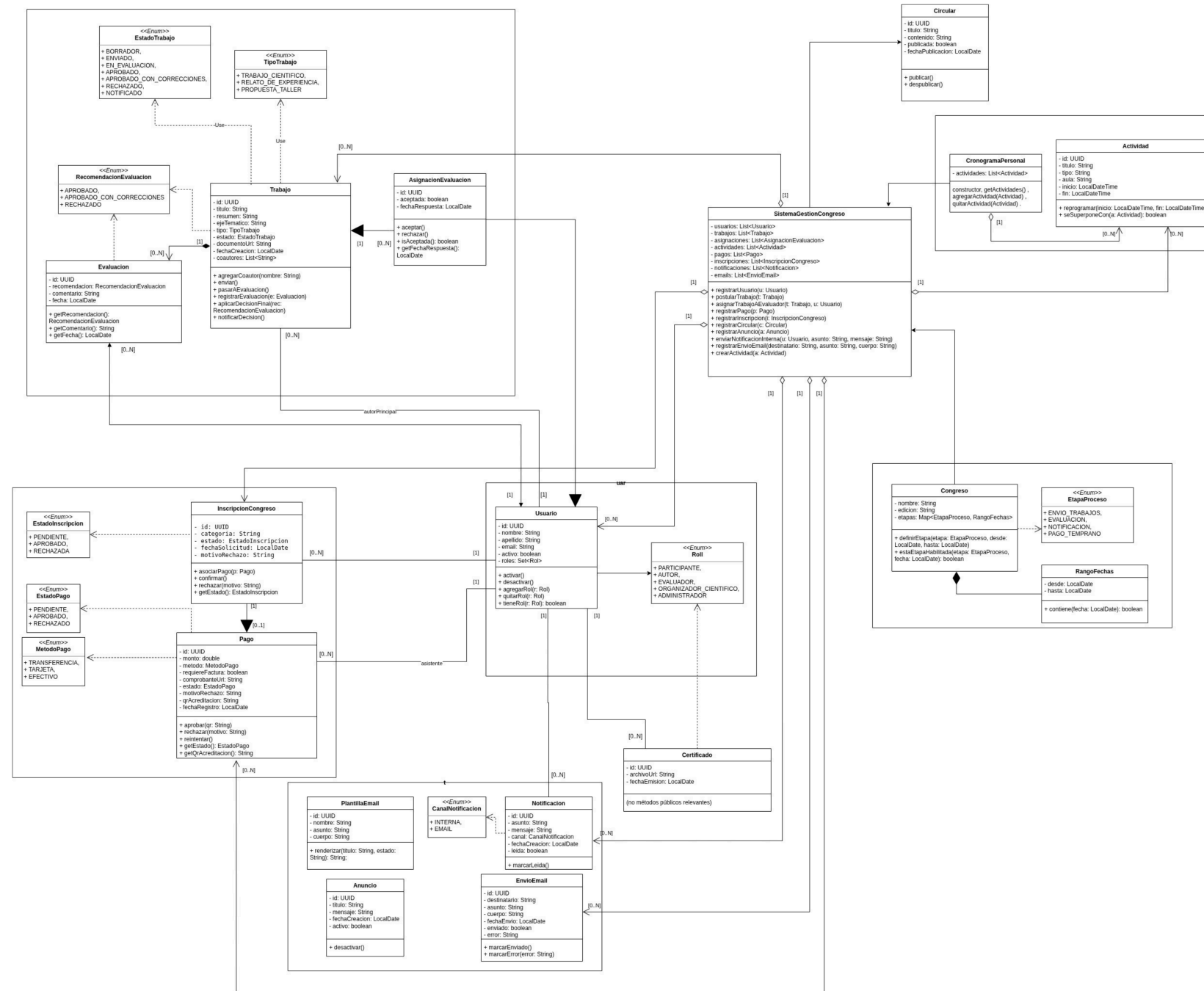
DAO = Data Access Object: es un **patrón de diseño** (no “magia de JPA”). En el código es una clase —o interfaz + implementación— cuya **única** responsabilidad es el acceso a datos de **una** entidad: guardar / buscar / actualizar / borrar.

- `Usuario` = datos y reglas del usuario (modelo E2).
- `UsuarioDAO` + `UsuarioDAOImpl` = cómo entra y sale de MySQL (E3).
- **JPA** (`@Entity` , `EntityManager`) = la herramienta que usa el DAO para mapear objeto ↔ fila.

Por eso el modelo nos llevó a DAOs: ya teníamos objetos bien cortados; el DAO es el “portero” de cada uno hacia la base, sin ensuciar el diagrama de negocio.

Camino del diseño

HU → clase del diagrama → POJO → entidad JPA → tabla MySQL → DAO (CRUD de esa entidad) → Factory (un solo lugar crea el DAO) → en E4, CDI inyecta esas dependencias.



Del diagrama a la persistencia (idea fija)

una caja → un DAO

Cuando hagan zoom en el diagrama, piensen esto: **cada clase con `id` es candidata a tabla y a DAO.**

EN EL MODELO (E2)	EN LA PERSISTENCIA (E3)	QUIÉN LO HACE
Usuario	tabla <code>usuario</code>	<code>UsuarioDAO</code>
Trabajo	tabla <code>trabajo</code>	<code>TrabajoDAO</code>
Pago	tabla <code>pago</code>	<code>PagoDAO</code>
AsignacionEvaluacion	su tabla	<code>AsignacionEvaluacionDAO</code>
Evaluacion	su tabla	<code>EvaluacionDAO</code>
CRUD repetido (save/find/delete)	código compartido	<code>GenericDAO</code> / <code>AbstractJpaDAO</code>
“¿quién me da el DAO?”	un solo lugar de creación	<code>DAOFactory</code> (Factory)

Nuestro diseño de objetos partió el dominio en entidades. Eso hizo natural el patrón **DAO**: cada entidad se persiste por su cuenta, sin meter MySQL dentro de las clases del negocio. **JPA** convierte el objeto en una fila de la tabla (y al revés). El **DAO** es quien pide ese guardar/buscar. El **Factory** es el único lugar que crea los DAOs: el resto del sistema no tiene que hacer `new UsuarioDAOImpl()` en cada pantalla.

Capa de persistencia (+ primer REST)

Producto: JPA + DAO + Factory · MySQL

Objetivo

Tomar el modelo de la Entrega 2 y hacer que **sobreviva al reinicio** en MySQL (`jyaa2026_bd1`): mismas clases → tablas; mismas relaciones → FK.

Qué aplicamos (directo del diseño E2)

JPA/Hibernate: cada POJO con `id` → `@Entity`. **Patrón DAO**: un DAO por entidad (`UsuarioDAO`, `TrabajoDAO` ...) sobre `GenericDAO` → `AbstractJpaDAO`. **DAOFactory**: punto único para pedir DAOs (aún sin CDI). Arranque con `ServletContextListener`. ABM en `/test-persistencia`.

Cómo estaba armado el REST al principio

Empezamos a exponer una API con **JAX-RS / Jersey**, pero los recursos **instanciaban los DAOs a mano** (`DAOFactory` / `new`). Eso **mezclaba capas**: la capa web hablaba directo con la persistencia, sin servicios ni CDI.

Dificultades

- ① Listar entidades a mano en `persistence.xml` (Tomcat no escanea como un servidor Jakarta EE completo).
- ② PDFs: pasar el `@Lob` a **LONGBLOB**.
- ③ Migraciones propias (`SchemaMigration`) porque `hbm2ddl` no alcanza.
- ④ La mezcla de capas funcionaba, pero no escalaba: era el problema que resolvimos en la Entrega 4.



CAPTURA 3 — Persistencia

`/test-persistencia` o tablas en phpMyAdmin. Guardar como `img/e3-persistencia.png`.

Rama `entrega-3`: la base queda en MySQL; el REST todavía sin capas limpias.

Servicios REST con separación en capas (CDI)

Producto: API + CDI + JWT + Swagger

Objetivo

Aplicar **SOLID**, sobre todo el **DIP**: el alto nivel depende de abstracciones. Eso se concreta con **CDI** (`@Inject`): dejamos de crear DAOs a mano y separamos Resource → Service → DAO.

Qué cambió respecto de la Entrega 3

En E3 el patrón creacional era `DAOFactory`. Acá el contenedor **CDI / Weld** reemplaza ese `new`: **Resource** → **Service** → **DAO**, con `EntityManagerProducer` (`@Produces` / `@Disposes`) por request. SRP: el Resource no hace SQL; el DAO no arma JSON.

Qué más sumamos

Jersey arma las URLs de la API · **DTOs** (no devolvemos entidades JPA crudas) · **JWT** para el login · filtros CORS · **Swagger/OpenAPI** para documentar y probar · `ExceptionMappers`.

Dificultad

Hacer convivir **Jersey + CDI en Tomcat**: sin `jersey-cdi1x-servlet` / `jersey-hk2` y `beans.xml`, las inyecciones llegaban en `null`. **Cómo lo resolvimos**: el contenedor CDI crea e inyecta; nosotros dejamos de instanciar a mano.



CAPTURA 4 — Swagger UI

Recursos y un `POST /api/login`. Guardar como `img/e4-swagger.png`.

Rama `entrega-4`: la API queda en capas, documentada y usable por el front.

Sesión con JWT: por qué no usamos `HttpSession`

Cómo funciona

- `POST /api/login` devuelve un **JWT firmado con HS256**, válido 4 horas.
- El cliente lo manda en `Authorization: Bearer ...` en cada pedido.
- `JwtAuthFilter` lo valida y arma un `AuthenticatedUser` para el recurso.
- Rutas públicas declaradas: login, registro, `health`, programa, circulares y OpenAPI.
- En el cliente: `sessionStorage` + cierre por inactividad a los 30 minutos.

El detalle que aprendimos peleándonos

Un token es una foto del momento en que se emitió. Si el administrador te quita un rol o te inhabilita la cuenta, el token viejo **seguiría siendo válido** durante horas.

Nuestra solución: el filtro valida la firma del token, pero vuelve a leer **roles y estado activo desde la base** en cada request. Así, inhabilitar una cuenta la expulsa al instante, sin esperar el vencimiento.

Por qué JWT y no `HttpSession`: la API tiene que poder servir a un cliente Angular alojado en otro origen y a un backend replicado en contenedores. Sin estado en el servidor, el escalado y el CORS se simplifican.

Capa de presentación e integración

Producto: SPA Angular 19 contra la API real

Objetivo

Reemplazar el prototipo de la Entrega 1 (datos en el navegador) por un cliente real que consuma la API de la Entrega 4 y respete los permisos de cada rol.

Qué aplicamos

SPA Angular = pantallas en el navegador que piden datos al servidor por la API. Componentes standalone, rutas, **guards** por rol, interceptor JWT, formularios reactivos, `HttpClient`.

Herramientas

Angular 19, RxJS, Leaflet (mapa), proxy de desarrollo hacia el servidor de la cátedra.

Dificultades

- ① **CORS** (proxy + filtros en el backend).
- ② `inject()` mal usado en `ngOnInit` (NG0203).
- ③ Bundle que superaba el presupuesto del build.
- ④ Mapa Leaflet que parpadeaba al recrearse.



CAPTURA 5 — Angular + Network

Panel con DevTools → `/api/...` + `Authorization`. Guardar como `img/e5-angular.png`.

Rama `entrega-5`: la vista habla con la API en capas de la Entrega 4.

Sistema completo

Producto: aplicación en producción, ciclo de vida completo del congreso

Objetivo

Cerrar el ciclo completo — de la inscripción al certificado — y dejarlo desplegado y usable por los organizadores.

Qué sumamos en esta etapa

Aranceles configurables por categoría con QR · recibo de caja correlativo y arqueo diario · factura al participante · cupos de evaluador por eje con liberación automática · desempate 1–1 que dispara un tercer evaluador · catálogos editables por el comité · programa con aulas georreferenciadas y control de cupo · circulares y notificaciones por plantilla (in-app + email) · checklist pre-congreso · cierre del congreso que emite los certificados.

Integración final

Un **Dockerfile multi-stage** compila Angular, lo copia dentro de `src/main/webapp`, empaqueta el WAR con Maven y lo despliega en Tomcat 10 / JDK 21. Frontend y API terminan en el **mismo origen**: en producción ya no hace falta CORS.

Dificultad

El contenedor a veces se desplegaba con un frontend viejo por la caché de Docker. **Cómo la resolvimos:** build con `--no-cache --pull` y, sobre todo, **pasos de verificación dentro del propio Dockerfile**: si el WAR no contiene el `main-*.js` y los estilos esperados, la imagen no se construye. El error se detecta en el pipeline, no en la demo.



CAPTURA 6 — Sistema en producción

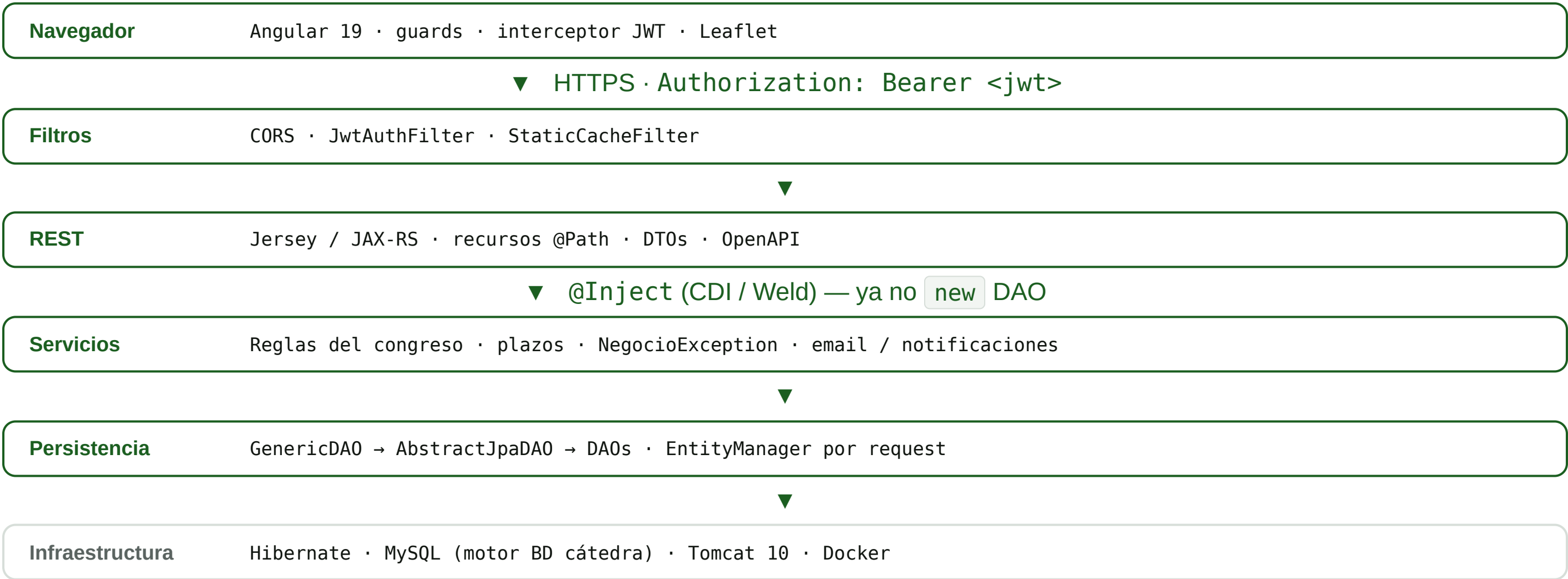
Home pública en `grupo1.jyaa-ci.linti.unlp.edu.ar` o el programa del congreso con el mapa de aulas. Guardar como `img/e6-produccion.png`.

Desplegado y accesible: cada push a `main` lo reconstruye.

La arquitectura que resultó de las seis entregas

Un solo WAR

Primero persistimos y hablamos a los DAOs desde el REST (capas mezcladas). En la Entrega 4 separamos con CDI. Hoy quedó así:



Cada tema de la cátedra, en un archivo concreto del proyecto

14 clases teóricas

TEORÍA	DÓNDE VIVE EN EL PROYECTO	TEORÍA	DÓNDE VIVE EN EL PROYECTO
Arquitectura server-side y Servlets	PersistenciaTestServlet · web.xml · WAR en Tomcat	Inyección de dependencias (CDI/Weld)	Entrega 4: Resource → Service → DAO · EntityManagerProducer · beans.xml (en E3 aún se instanciaban DAOs a mano)
Sesiones	JWT 4 h + idle 30 min (en vez de HttpSession)	REST	JaxRsApplication · @Path · verbos · ExceptionMappers
ServletContext y Listeners	JpaBootstrapListener → EMF + datos iniciales	Swagger / OpenAPI	OpenApiResource · @Tag · @Operation
Filtros	JwtAuthFilter · StaticCacheFilter · filtros CORS	CORS	CorsRequestFilter @PreMatching (preflight OPTIONS)
Maven	pom.xml · packaging war · build multi-stage	Angular — parte 1	standalone components · app.routes.ts · servicios
JDBC, Datasource y patrón DAO	GenericDAO → AbstractJpaDAO → 23 DAOs	Angular — parte 2	HttpClient tipado · formularios reactivos · RxJS
JPA	23 @Entity · @ElementCollection · JPQL paginado	Angular — parte 3	guards de rol · interceptor JWT · build de producción

No quedó ningún tema de la cursada sin usar: el trabajo final los integró a todos en un mismo despliegue.

03

Cómo trabajamos

La organización interna del grupo y las herramientas que la sostuvieron.

Nos repartimos por flujo del congreso, no por capa

3 integrantes

Al principio dividimos por capas —uno el backend, otro el front— y chocábamos todo el tiempo en los mismos archivos. A partir de la Entrega 3 cambiamos: **cada uno se hizo dueño de un flujo completo**, de la entidad JPA a la pantalla Angular.

Benjamín

Inscripciones, pagos, aranceles, arqueo y facturación · gestión de usuarios · el grueso del modelo JPA y los recursos REST.

157 commits

Alcides

Flujo académico: envío de trabajos, comité, asignaciones, dictámenes y desempates · programa, aulas, cronogramas y mapas.

42 commits

Lucas

Autenticación y roles · certificados automáticos · circulares y notificaciones · despliegue, pipeline y pruebas de integración.

8 commits

Lo que funcionó

Ser dueño de un flujo de punta a punta bajó los conflictos de merge casi a cero y hizo que cada uno tuviera que aprender **todas** las capas.

Lo que nos costó

Los cambios en `Usuario` y en la configuración del congreso tocan a los tres. Los coordinamos antes de codear y los integramos primero.

Git, GitLab, Docker y despliegue continuo

push a main → app actualizada

- **Git con dos remotos:** GitHub para el trabajo diario del grupo y **GitLab de la cátedra** como remoto de entrega y despliegue.
- **Una rama por entrega** (entrega-3 ... entrega-5) más ramas de feature por persona; `main` siempre desplegable.
- **Docker multi-stage:** etapa Node que compila Angular, etapa Maven que arma el WAR, imagen final Tomcat 10 / JDK 21.
- **GitLab CI** con runner `jyaa` : construye, detiene y elimina el contenedor anterior y levanta el nuevo detrás de Traefik con TLS automático.
- **Verificación en el build:** el Dockerfile comprueba el contenido del WAR antes de publicar la imagen.

Git

GitLab CI

Docker

Traefik + TLS

Maven

Eclipse / VS Code

phpMyAdmin

Swagger UI

Postman / DevTools

El pipeline, en cuatro pasos

- **build** — `docker buildx build --no-cache :`
Angular + WAR frescos y verificados
- **test** — se detiene el contenedor `grupo1` anterior
- **test** — se elimina el contenedor viejo
- **deploy** — `docker run` con las etiquetas de Traefik para `grupo1.jyaa-ci.linti.unlp.edu.ar`



CAPTURA 7 — Pipeline en verde

Los cuatro jobs de GitLab CI completados. Guardar como `img/ci-pipeline.png`.

04

Demo y reflexión

El sistema funcionando de punta a punta, y qué nos llevamos del proceso.

Un trabajo, de principio a fin

6 min · grupo1.jyaa-ci.linti.unlp.edu.ar

- **1 · Público** — home, programa del congreso con el mapa de aulas y circulares, sin iniciar sesión.
- **2 · Asistente** — registro, inscripción por pasos, pago por transferencia con comprobante.
- **3 · Administración** — aprueba el pago, se emite el recibo correlativo y llega la notificación.
- **4 · Autor** — carga un trabajo con coautores, sube el PDF y lo envía.
- **5 · Comité** — precheck, asigna dos evaluadores respetando el cupo del eje.
- **6 · Evaluador** — acepta la asignación y emite el dictamen.
- **7 · Autor** — ve la devolución en su panel.
- **8 · Administración** — cierra el congreso y se habilitan los certificados.

Lo que queremos que se vea

No las pantallas sueltas, sino que **una acción de un rol cambia lo que ve otro rol**, en la misma base de datos y en tiempo real.

Plan B

Si la red falla: video de respaldo del recorrido completo en `img/demo-respaldo.mp4` y capturas de cada paso.

Preparado de antemano

Sesiones abiertas en pestañas separadas por rol, congreso con etapas vigentes y un trabajo listo para enviar.

Las cinco dificultades que más nos enseñaron

y cómo las resolvimos

DIFICULTAD	QUÉ PASABA	CÓMO LA RESOLVIMOS
Capas mezcladas → CDI	En la Entrega 3 el REST instanciaba DAOs a mano (<code>DAOFactory</code> / <code>new</code>). En la 4, al meter CDI en Tomcat, las inyecciones llegaban en <code>null</code> .	Weld + puentes Jersey-CDI + <code>beans.xml</code> ; Resource → Service → DAO con <code>@Inject</code> y <code>EntityManagerProducer</code> .
CORS	El navegador bloqueaba las llamadas del Angular local al backend desplegado.	Proxy de Angular para desarrollo y filtros CORS con preflight para el resto. En producción, mismo origen: el problema desaparece.
Esquema de base	<code>hbm2ddl.auto=update</code> agrega columnas pero nunca las corrige; los PDF no entraban y un campo quedó corto.	Clase <code>SchemaMigration</code> con SQL propio que corre al arrancar la aplicación, más <code>LONGLOB</code> para los documentos.
Deploy con front viejo	La caché de Docker publicaba un WAR con un Angular desactualizado.	<code>--no-cache --pull</code> y verificaciones del contenido del WAR dentro del Dockerfile: el pipeline falla antes de desplegar.
Trabajo en paralelo	Conflictos constantes al dividirnos por capas, y <code>node_modules</code> versionado por error.	Reparto por flujo funcional, <code>.gitignore</code> saneado y una rama por entrega que sólo se integra con el pipeline en verde.

Qué nos llevamos

2 min

Las capas no son burocracia

Cuando cambiamos el frontend entero de React a Angular, el backend no se tocó. Ese día entendimos para qué sirve separar en capas.

El modelo de datos es la decisión más cara

Convertir la asignación de evaluadores en una entidad propia costó una tarde en la Entrega 2 y nos ahorró semanas en la 4 y la 6.

Configurable le gana a hardcodeado

Etapas, catálogos, aranceles y cupos los define el organizador. Si estuvieran en el código, cada edición del congreso pediría un despliegue.

Desplegar temprano y seguido

Tener CI desde la Entrega 3 convirtió “anda en mi máquina” en un problema de cinco minutos en vez de una crisis la noche anterior a entregar.

Programar en equipo es coordinar, no repartir

Git resuelve los archivos; las decisiones sobre el dominio compartido hay que conversarlas antes de escribirlas.

Si tuviéramos una entrega más

Hash de contraseñas con BCrypt, autorización declarativa con `@RolesAllowed`, Bean Validation en los DTOs y pruebas automatizadas dentro del pipeline.

Gracias. ¿Preguntas?

Sistema de gestión del V Congreso Argentino de Agroecología

Aplicación: grupo1.jyaa-ci.linti.unlp.edu.ar **API:** </api/health> · **OpenAPI:** </api/openapi.json>

Repositorio: [GitLab de la cátedra — grupo 1](#)